Exercise: Mobile Testing

Playwright Mobile Web Testing · TodoMVC · Responsive layouts · Visual regression

1. Before you start

You set the project up at the end of **Exercise I** — the folder created with `npm init playwright@latest`, the "Playwright Test for VSCode" extension by Microsoft, and a `playwright.config.js` switched to the Pixel 7 device profile. Open that folder in VS Code and carry on from there.

If you skipped that part or are starting fresh, it takes about two minutes: create an empty folder, run `npm init playwright@latest` in a VS Code terminal (JavaScript, `tests` folder, no GitHub Actions, yes to installing browsers), install the VS Code extension, then open `playwright.config.js` and replace the generated `projects` array with a single mobile project. Leave the rest of the file as it is:

```
// playwright.config.js — replace the generated projects array with:
projects: [
  {
    name: 'Mobile Chrome',
    use: { ...devices['Pixel 7'] },
  },
],
```

Every test then runs against a Pixel 7-sized viewport, with touch enabled and a matching user agent — no cloud account, no device farm, no Appium server.

The generator also created a tests/example.spec.js file with two sample tests. Delete it — otherwise it runs alongside everything you write from here on, and the report fills up with results you did not author.

> 💡 **import, not require**
> The generated project uses ES modules, so every test file starts with an `import` line rather than `require(...)`. You will still see the older `require` style in tutorials online — the two cannot be mixed in one project, so stick with `import` throughout.

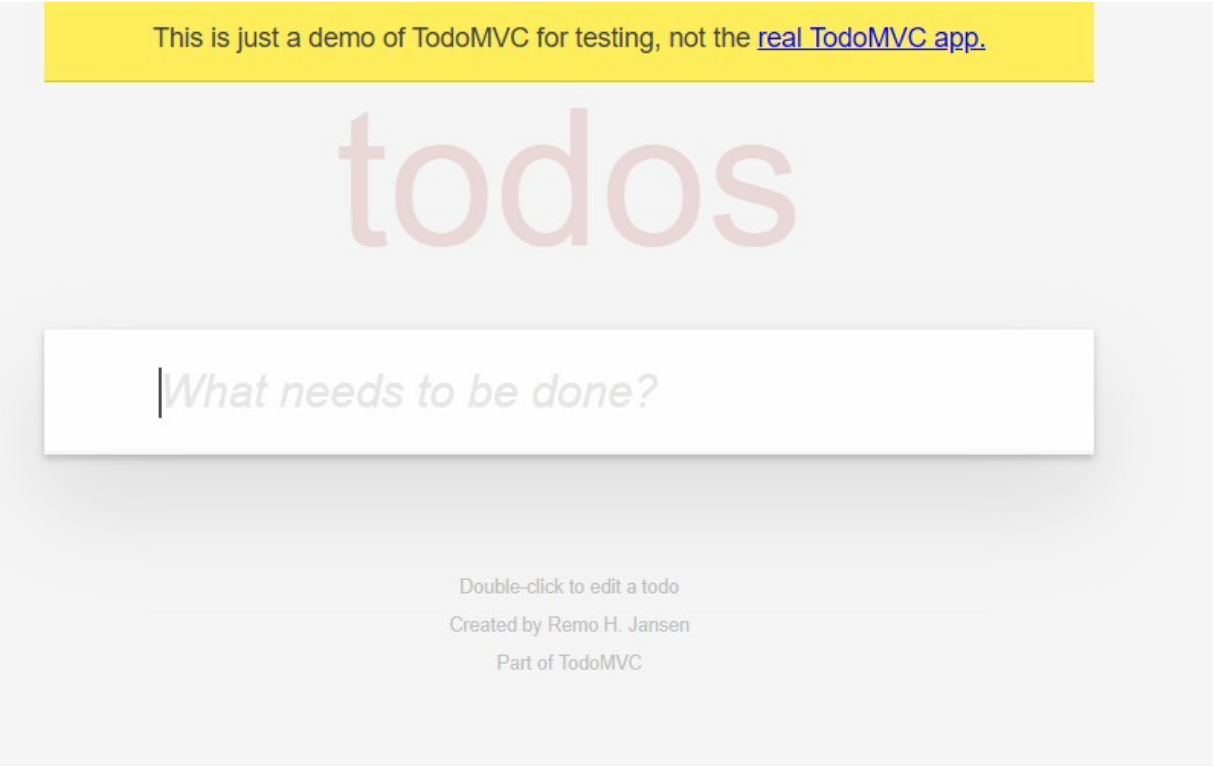
1.1. Finding locators

Unlike Appium, Playwright has no separate Inspector app. Instead:

- **Built-in locators** — `getByRole`, `getByPlaceholder`, `getByText`. These read the page's accessibility tree, so they are usually stable without hunting for an id.
- **Codegen (recording)** — click "Record new" in the Testing sidebar, or run `npx playwright codegen https://demo.playwright.dev/todomvc/`. It opens a real browser, records your clicks, and writes the test code.
- **Pick locator tool** — hover over any element in a running browser to see the best available locator, then click to copy it.

2. TodoMVC (warm-up)

We start with Playwright's own demo app: https://demo.playwright.dev/todomvc/



It is the TodoMVC reference app used throughout Playwright's documentation — small, stable, and ideal for a first mobile web test.

Create `tests/todomvc.spec.js`:

```
import { test, expect } from '@playwright/test';

test.beforeEach(async ({ page }) => {
  await page.goto('https://demo.playwright.dev/todomvc/');
});
```

Every test starts on a clean page. Notice there is no explicit wait anywhere in this file — Playwright's **auto-waiting** retries each action until the element is actionable.

2.1. Add todos and mark one complete

```
test('add todos and mark one complete', async ({ page }) => {
  const newTodo = page.getByPlaceholder('What needs to be done?');

  await newTodo.fill('Buy groceries');
  await newTodo.press('Enter');
  await newTodo.fill('Write Playwright tests');
  await newTodo.press('Enter');

  const items = page.getByTestId('todo-item');
  await expect(items).toHaveCount(2);

  await items.nth(0).getByRole('checkbox').check();
  await expect(items.nth(0)).toHaveClass(/completed/);
});
```

2.2. Edit an existing todo

TodoMVC enters edit mode on a double-click of the item's text, revealing an editable field. Type the new text and press Enter to save:

```
test('edit an existing todo', async ({ page }) => {
  const newTodo = page.getByPlaceholder('What needs to be done?');
  await newTodo.fill('Original task text');
  await newTodo.press('Enter');

  const todoItem = page.getByTestId('todo-item').first();
  await todoItem.dblclick();

  const editBox = todoItem.getByRole('textbox', { name: 'Edit' });
  await editBox.fill('Updated task text');
  await editBox.press('Enter');

  await expect(todoItem).toHaveText('Updated task text');
});
```

> 💡 **Escape cancels, Enter saves**
> While a todo is in edit mode, Enter commits the new text and Escape throws it away and restores the original. The test above only covers Enter. Open the app by hand, double-click a todo, type something, press Escape, and watch the old text come back — then you know what the other half of this behaviour looks like before you decide whether it is worth a test of its own.

2.3. Filter Active and Completed

```
test('filter active and completed', async ({ page }) => {
  const newTodo = page.getByPlaceholder('What needs to be done?');
  await newTodo.fill('Task A'); await newTodo.press('Enter');
  await newTodo.fill('Task B'); await newTodo.press('Enter');

  await page.getByTestId('todo-item').nth(0).getByRole('checkbox').check();

  await page.getByRole('link', { name: 'Active' }).click();
  await expect(page.getByTestId('todo-item')).toHaveCount(1);

  await page.getByRole('link', { name: 'Completed' }).click();
  await expect(page.getByTestId('todo-item')).toHaveCount(1);
});
```

2.4. Clear completed

```
test('clear completed', async ({ page }) => {
  const newTodo = page.getByPlaceholder('What needs to be done?');
  await newTodo.fill('Temporary task'); await newTodo.press('Enter');

  await page.getByTestId('todo-item').getByRole('checkbox').check();
  await page.getByRole('button', { name: 'Clear completed' }).click();

  await expect(page.getByTestId('todo-item')).toHaveCount(0);
});
```

> 💡 **data-testid vs resource-id**
> In Appium you located elements by `resource-id`, an attribute Android puts in the view hierarchy. The web has no equivalent of its own, so teams add one: a `data-testid` attribute written into the HTML for the sole purpose of being found by tests. `getByTestId('todo-item')` reads exactly that attribute.

> The difference matters. `resource-id` is there whether anyone planned for testing or not; `data-testid` exists only because a developer put it there. TodoMVC ships them, which is why these tests work. On a project that does not, you either add them yourself or fall back to `getByRole` and `getByText`.

3. Running and debugging

- In VS Code's Testing sidebar, click the play button next to a test — or run everything from the terminal:

```
npx playwright test
```

- Tests run headless by default. To watch the browser, use `--headed`:

```
npx playwright test --headed
```

- Even headed, tests finish almost instantly — Playwright never waits longer than it has to. To step through one action at a time, run with `--debug`. It opens the Playwright Inspector next to the browser, and you advance manually:

```
npx playwright test --debug
```

- To slow everything down without stopping, add `slowMo` to the project in `playwright.config.js`. There is no command-line flag for it:

```
{
  name: 'Mobile Chrome',
  use: {
    ...devices['Pixel 7'],
    launchOptions: { slowMo: 1000 },
  },
},
```

- Open the HTML report for results, screenshots, and timings:

```
npx playwright show-report
```

- **UI Mode** is the most comfortable way to work — a watch-mode interface with a time-travel timeline of every test:

```
npx playwright test --ui
```

UI Mode opens empty — click a test in the list on the left to load its steps, then click any step to see a snapshot of the page at that moment.

4. Responsive layout

TodoMVC looks nearly the same at every screen size, so it cannot show you what mobile web testing is actually for. For that we switch to a site with a real responsive layout:
`https://playwright.dev` — Playwright's own documentation site.

On a desktop viewport its navigation bar shows the links (Docs, API, and so on) laid out horizontally. Below a breakpoint, those links collapse behind a hamburger button. Same HTML, same URL — a completely different interface. Catching a break in either one means testing at both sizes.

4.1. Add a desktop project to the config

To compare the two layouts, you need two projects — one phone-sized, one desktop-sized. Add a second entry to the `projects` array in `playwright.config.js`:

```
projects: [
  {
    name: 'Mobile Chrome',
    use: { ...devices['Pixel 7'] },
  },
  {
    name: 'Desktop Chrome',
    use: { ...devices['Desktop Chrome'] },
  },
],
```

Now `npx playwright test` runs every test twice — once per project — and the report labels each result with the project it ran under. You can also target one explicitly with `npx playwright test --project="Mobile Chrome"`.
This applies to the TodoMVC tests too — from now on they run under both projects, so the result count doubles. They pass either way, since TodoMVC barely changes with screen size.

4.2. Testing the responsive breakpoint

Create `tests/responsive.spec.js`. Both tests live here, and both run under both projects — but each one only makes sense in one of them, so each starts by skipping itself in the wrong project:

```
import { test, expect } from '@playwright/test';


test('hamburger opens the navigation', async ({ page, isMobile }) => {
  test.skip(!isMobile, 'mobile layout only');


  await page.goto('https://playwright.dev/');
  const menuButton = page.getByRole('button', { name: 'Toggle navigation
bar' });
  await expect(menuButton).toBeVisible();


  // tap() instead of click() — this is a touch device
  await menuButton.tap();
  await expect(page.getByRole('link', { name:
'Docs' }).first()).toBeVisible();
});


test('navigation links are visible without a menu', async ({ page, isMobile
}) => {
  test.skip(isMobile, 'desktop layout only');
```

```
  await page.goto('https://playwright.dev/');
  await expect(
    page.getByRole('button', { name: 'Toggle navigation bar' })
  ).toBeHidden();
  await expect(page.getByRole('link', { name:
'Docs' }).first()).toBeVisible();
});
```

`isMobile` is a fixture Playwright fills in from the project's device profile - `true` under Mobile Chrome, `false` under Desktop Chrome. `test.skip()` with a condition leaves the test alone when the condition holds, and the report marks it as skipped rather than passed, so you can see at a glance which layout each result belongs to.
Note `menuButton.tap()` rather than `click()`. On a touch-enabled context `tap()` dispatches real touch events, which is what a phone would fire — the direct conceptual counterpart of the gestures Appium automates on a native app.

> ⚠️ **tap() requires hasTouch**
> Calling `tap()` on a context without touch enabled throws an error telling you to set `hasTouch: true`. The device descriptors set it for you — here it comes from `devices['Pixel 7']` in the Mobile Chrome project — but if you write your own `use` block by hand, remember it.

Run it and you get four results from two tests — each test once per project, one of them skipped each time:

```
Running 4 tests using 4 workers

  -  1 [Mobile Chrome] › tests\responsive.spec.js:14:5 › navigation links are visible without a menu
  -  2 [Desktop Chrome] › tests\responsive.spec.js:3:5 › hamburger opens the navigation
  ✓  3 [Mobile Chrome] › tests\responsive.spec.js:3:5 › hamburger opens the navigation (6.3s)
  ✓  4 [Desktop Chrome] › tests\responsive.spec.js:14:5 › navigation links are visible without a menu (4.1s)

  2 skipped
  2 passed (7.7s)
```

> ⚠️ **Device options cannot go in a describe block**
> You might expect to group these with `test.describe()` and call `test.use({ ...devices['Pixel 7'] })` inside each group. Playwright rejects that outright, before running anything:
> `Cannot use({ defaultBrowserType }) in a describe group, because it forces a new worker.`
> Device profiles change browser-level settings, and a describe block cannot start the new worker that requires. Such options belong either at the top level of a file — applying to every test in it — or, as here, in `playwright.config.js`.

5. Visual regression testing

Functional tests check that things work. They will happily pass while a stylesheet change pushes a button off screen, a font fails to load, or a mobile layout collapses into an unreadable mess. Visual regression testing catches exactly those.

Playwright has this built in through `toHaveScreenshot()`. On the first run it saves a reference image (a *baseline*, sometimes called a golden image). On every run after that it takes a fresh screenshot and compares it pixel by pixel against the baseline.

5.1. Your first visual test

Create `tests/visual.spec.js`:

```
import { test, expect } from '@playwright/test';


test('playwright.dev homepage looks right on mobile', async ({ page }) => {
  await page.goto('https://playwright.dev/');
  await expect(page).toHaveScreenshot('homepage-mobile.png');
});
```

Run it. **The first run fails** — and that is intentional. Playwright reports something like:

```
Error: A snapshot doesn't exist at
tests/visual.spec.js-snapshots/homepage-mobile-Mobile-Chrome-win32.png,
writing actual.
```

There was no baseline yet, so Playwright created one and failed the test to make you look at it. Open the generated PNG and check it actually shows what you expect. Run the test a second time and it passes — the fresh screenshot now matches the file on disk. The name encodes the project and platform (`Mobile-Chrome-win32`) because rendering differs between browsers and operating systems, so each combination needs its own baseline.

That also means one baseline per project: with both Mobile Chrome and Desktop Chrome configured, this single test writes two images and fails twice on the first run. Both are expected.

> 💡 **Baselines belong in version control**
> Commit the -snapshots folder alongside your tests. Reviewing changes to those images in a pull request is a large part of what makes visual testing worth the trouble — a diff you can see is easier to judge than a diff you have to imagine.

When a visual change is intentional rather than a regression, you re-record the baseline instead of fixing the test:

```
npx playwright test --update-snapshots
```

6. What this exercise adds

- A second, genuinely different automation model — declarative locators and auto-waiting, instead of explicit capabilities, driver sessions, and implicit waits.
- Zero infrastructure — no AWS account, no device farm, no Appium server, no trial minutes to budget. Everything runs locally in seconds.
- Layout as a testable property — responsive tests and visual baselines cover a category of bug that functional tests, in either tool, will pass straight through.
- A concrete answer to "when would I use Playwright instead of Appium" — the moment the thing under test is a website or a WebView, rather than a native screen.